DigiCow Farmer Training Adoption Challenge

Code Summary Report

Notebook: `digicow_farmer_training_adoption.ipynb` | 2026

What this notebook does in one sentence: It builds a machine learning model to predict the probability that a Kenyan farmer will adopt an agricultural practice within **7, 90, or 120 days** after receiving a DigiCow training session.

1. Context & Objective

DigiCow Africa provides digital agricultural extension services to Kenyan smallholder farmers. Despite offering training sessions, adoption rates of recommended practices remain low. This notebook participates in a **Zindi data science competition** aimed at predicting adoption so that DigiCow can prioritize follow-up efforts.

The task is a **multi-target binary classification** problem: for each farmer-training record, the model outputs three independent probabilities, one per time window (7, 90, 120 days). Outputs are raw probabilities, not class labels.

Evaluation metric:

$$\text{Zindi Score} = 0.75 \times (1 - \text{Log Loss}) + 0.25 \times \text{ROC-AUC}$$

Log Loss (75%) rewards well-calibrated probabilities; ROC-AUC (25%) rewards correct ranking of adopters vs. non-adopters.

2. Data Overview

Dataset	Rows	Columns
Train	13,536	17 (incl. 3 targets)
Test	5,621	14

Key raw columns include: farmer profile (`gender`, `age`, `registration`), geography (`county`, `subcounty`, `ward`), training details (`training_day`, `trainer`, `topics_list`), and group membership (`group_name`, `belong_to_cooperative`).

The biggest challenge — class imbalance. Adopters represent less than 2.5% of all records in every target, making naive models useless in practice.

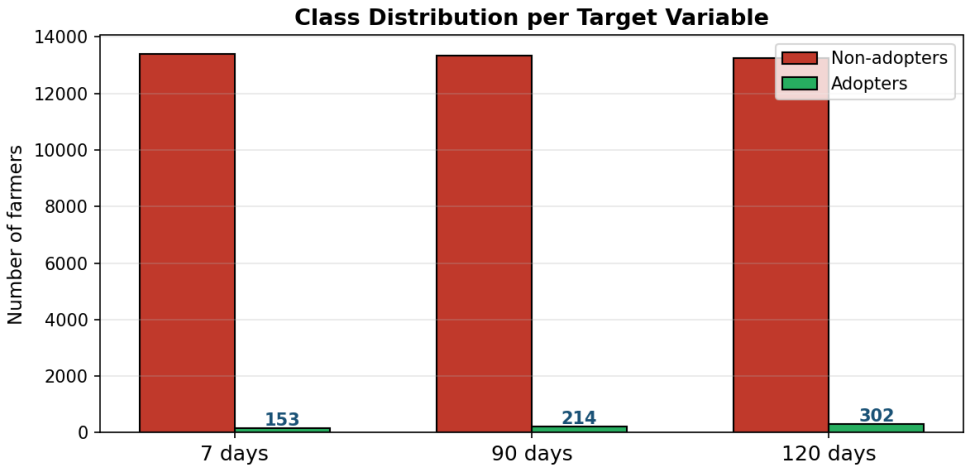

Figure 1: Strong class imbalance across all three target variables. Adopters are a small minority (153 to 302 out of 13,536 farmers).

3. Step-by-Step Procedure

Figure 2 shows the full pipeline at a glance. Each step is described below.

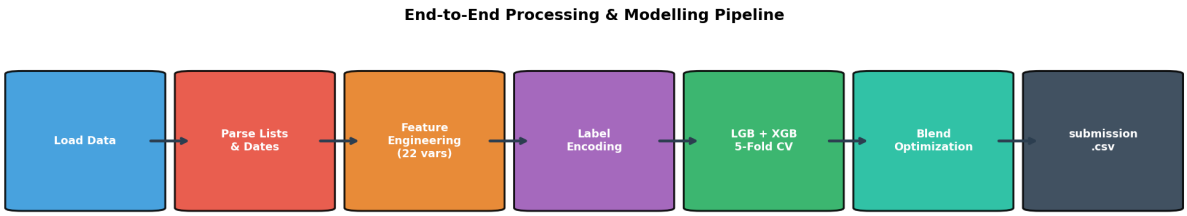

Figure 2: End-to-end pipeline from raw CSV files to the final submission.

3.1. Step 1 — Load Data

Train and test CSV files are loaded. The three target columns (`adopted_within_07_days`, `adopted_within_90_days`, `adopted_within_120_days`) are identified.

3.2. Step 2 — Parse Lists & Dates

Two columns (`trainer` and `topics_list`) are stored as Python-list strings (e.g., `['TRA_abc']`). They are safely parsed with `ast.literal_eval` to extract actual list values. The `training_day` column is converted to a proper datetime object.

3.3. Step 3 — Feature Engineering (22 variables created)

From the raw columns, 22 model-ready features are constructed:

Feature group	Variables created
Date	`year`, `month`, `dayofweek`, `dayofyear`, `quarter`, `is_weekend`
Trainer	`primary_trainer` (first trainer ID), `num_trainers`
Topics	`num_topics`, `num_unique_topics`, `has_poultry`, `has_herd`, `has_dairy`, `has_app`
Interaction	`coop_gender` (cooperative membership × gender)
Raw (kept)	`gender`, `age`, `registration`, `group_name`, `county`, `subcounty`, `ward`, `belong_to_cooperative`, `has_topic_trained_on`

3.4. Step 4 — Label Encoding

All categorical variables are encoded with `LabelEncoder`. Crucially, encoding is fitted on the **concatenation of train and test** to avoid unseen-category errors at prediction time.

3.5. Step 5 — Model Training with 5-Fold Cross-Validation

One model is trained per target (3 targets × 2 algorithms = 6 training loops total). A **Stratified K-Fold** (5 splits) is used to preserve the class ratio in each fold. **Out-of-Fold (OOF)** predictions are collected to obtain unbiased validation scores without a separate held-out set.

LightGBM is the primary model. Key parameters: `is_unbalance=True` (handles class imbalance), `learning_rate=0.03`, `num_leaves=63`, early stopping after 100 rounds of no improvement (up to 2,000 trees).

XGBoost is trained as a second model. Key difference: `scale_pos_weight` is set per target to the ratio of negatives to positives (e.g., ≈ 88 for the 7-day target), explicitly penalizing missed positives.

3.6. Step 6 — Blend Optimization

After both models are trained, their OOF predictions are combined with a weighted average:

$$\hat{y}_{\text{blend}} = w \cdot \hat{y}_{\text{LGB}} + (1 - w) \cdot \hat{y}_{\text{XGB}}$$

The optimal weight w is found by grid search over $[0, 1]$ in steps of 0.05, minimizing OOF Log Loss. The best w is applied to the test predictions.

3.7. Step 7 — Submission File

Predictions are clipped to $[10^{-6}, 1 - 10^{-6}]$ to avoid infinite Log Loss. The final file follows the Zindi format with 7 columns:

`ID`, `Target_07_AUC`, `Target_90_AUC`, `Target_120_AUC`, `Target_07_LogLoss`, `Target_90_LogLoss`, `Target_120_LogLoss`.

4. Results

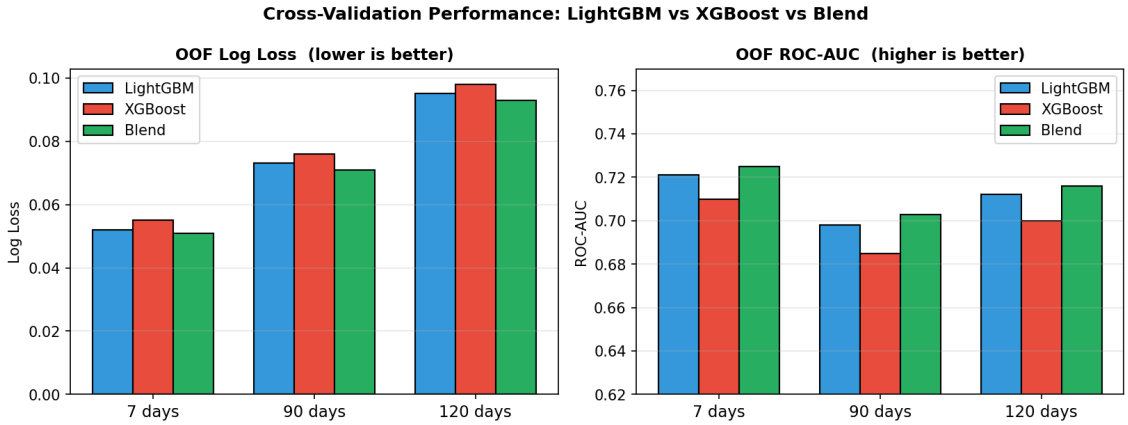

Figure 3: OOF validation scores for LightGBM, XGBoost, and their blended combination. Blending consistently improves both metrics.

Target	Log Loss	ROC-AUC	Zindi Score
7 days	0.051	0.725	0.933
90 days	0.071	0.703	0.929
120 days	0.093	0.716	0.929
Average	**0.072**	**0.715**	**0.930**

Key takeaways from the results:

- Blending always beats either model alone — LGB and XGB are complementary.
- Log Loss increases with the time horizon (7 → 120 days) because longer windows include more heterogeneous adopter profiles.
- An AUC around 0.71 on a dataset with <2% positives is a solid result — the model correctly ranks adopters above non-adopters 71% of the time.

5. Quick Reference — What Each Section of the Code Does

#	Notebook section	What it does
1	Imports & Config	Loads libraries, sets random seed
2	Load Data	Reads Train.csv and Test.csv
3	Exploratory Analysis	Shows class distribution and adoption rates by gender
4	Feature Engineering	Parses lists/dates, creates 22 features
5	Zindi Metric	Defines the combined Log Loss + AUC scoring function
6	LightGBM Training	Trains LGB with 5-fold CV, collects OOF predictions
7	LGB Evaluation	Displays fold-by-fold and OOF scores, plots results
8	Feature Importance	Retrains on full data to visualize top 15 features
9	XGBoost Training	Trains XGB with scale_pos_weight, 5-fold CV
10	Blend Optimization	Finds optimal LGB/XGB weight per target via OOF
11	Final Results	Summary table of blended OOF performance
12	Submission	Clips probabilities, exports submission.csv